## NAME

dbm – data base with extensible hashed indexing

## SYNOPSIS

```
import "github.com/forsyth/dbm"

type File struct {
    // symbols not exported
}

func Create(name string, perm int) (*File, error)
func Open(name string) (*File, error)
func OpenFile(name string, flag int, perms os.FileMode) (*File, error)

func (db *File) Fetch(key []byte) ([]byte, error)
func (db *File) Store(key []byte, data []byte., replace bool) error
func (db *File) Delete(key []byte) error

func (db *File) FirstKey() ([]byte, error)
func (db *File) NextKey(key []byte) ([]byte, error)

func (db *File) Flush() error
func (db *File) Reset()
func (db *File) IsReadOnly() bool
```

## DESCRIPTION

*Dbm* maintains key/content pairs in a data base. The functions will handle very large (a billion blocks) databases and will access a keyed item in one or two filesystem accesses.

*Key*s and *content*s are both represented by byte slices allowing arbitrary binary values.

The data base is stored in two files. One file is a directory containing a bit map and has `.dir` as its suffix. The second file contains all data and has `.pag` as its suffix. An application can access several databases at once, but must avoid concurrent operations on any one database (eg, by using a monitor goroutine to control access).

A database is created by `dbm.Create`, which accepts a file permission parameter *perm*, as described for `os.Create`; it creates the two files *name*`.dir` and *name*`.pag`. If successful, it returns a `File` pointer describing the database, which is open for reading and writing. `Create` will truncate an existing database. It returns an error if it cannot create the database for some reason.

`dbm.Open` opens an existing database in *name*`.dir` and *name*`.pag` for reading and writing. If successful, it returns a `File` pointer describing the database.

`dbm.OpenFile` is the generalised open file: it takes similar parameters to `os.OpenFile` for flags, and permissions if O_CREATE is passed in flags. It returns an error if the database cannot be opened successfully.

The remaining operations apply to an existing `*dbm.File` value *db*:

*db*`.Fetch(`*key*`)`
: Return the data stored under a *key*; nil is returned if the key is not in the database.

*db*`.Store(`*key, data, replace*`)`
: Store *data* under the given *key*. If the key already appears in the database, `Store` will return an error `ErrDuplicate` unless *replace* is true, causing it to replace the existing value by the new one.

*db*`.Delete(`*key*`)`
: *Key* and its associated value is removed from the database.

*db*`.FirstKey()`
: Return the first key in the database; return nil if the database is empty.

*db*.NextKey(*key*)
: Return the key following the given *key*, or an error if there is none.

*db.Flush*
: Ensure any data cached is written to the underlying files, and clear the cache.

*db.Reset*
: Discard any data cached from the file. The cache is write-through, so it is not necessary to flush the file before the application exits.

*db*.IsReadOnly()
: Return true if *db* was opened only for reading and writes are not allowed.

A datbase traversal using FirstKey and NextKey, must stop if the database is modified using Store or Delete.

**EXAMPLE**

A linear pass through all keys in a database may be made, in an (apparently) random order, by use of FirstKey and NextKey. This code will traverse the data base:

```
for key, err := db.FirstKey(); key != nil && err == nil; key, err = db.Ne
     data, err := db.Fetch(key)
     if err != nil {
          fmt.Fprintf(os.Stderr, "error fetching key %s: %s", string(key)
          break
     }
     fmt.Printf("%s0, string(key), string(data))
}
```

The order of keys presented by FirstKey and NextKey depends on a hashing function, not on anything interesting.

**DIAGNOSTICS**

| | |
|---|---|
| ErrCorrupt | corrupt data block |
| ErrDuplicate | key already exists |
| ErrNotFound | key not found |
| ErrNotPaired | corrupt database or internal error |
| ErrReadOnly | read-only file |
| ErrSplitNotPaired | corrupt database or internal error |
| ErrTooLarge | key/value too large |

**SOURCE**

github.com/forsyth/dbm/dbm.go
github.com/forsyth/dbm/cmd

**BUGS**

On some systems (notably Plan 9 but also some Unix systems), the .pag file might contain holes where no data block has ever been written so that its apparent size is about four times its actual content. These files cannot be copied by normal means (cp, cat) without filling in the holes.

The sum of the sizes of a key/content pair must not exceed the internal block size (currently 512 bytes). Moreover all key/content pairs that hash together must fit in a single block. Store will return an error in the event that a block fills with inseparable data.